

Suppliers & Customers → Parties

IMS (pharmacy inventory system) · Change documentation · 28 September 2026

In one sentence

The separate `suppliers` and `customers` tables were merged into a single `parties` table. Each party has a type, `Supplier`, `Customer` or `Both`, and purchases, sales, batches, returns and payments all link to a party through one `party_id` column.

Contents

1. Why we made this change
2. What “payment terms” means
3. Before and after: the data model
4. How party types work
5. Every file that changed
6. New API reference
7. User-interface changes
8. Database migration: what was done
9. Problem found and fixed afterwards
10. Things kept on purpose
11. Remaining steps and checklist
12. How to undo (rollback)

1. Why we made this change

Before, a business you *buy from* lived in `suppliers` and a business you *sell to* lived in `customers`. That caused three problems:

- **Duplicates.** A pharmacy or clinic that both sells to you and buys from you had to be entered twice, with two IDs and two copies of its phone, address and PAN.
- **Duplicated code.** There were two controllers, two sets of API routes, two validation files and two nearly identical pages doing the same job.
- **Awkward payments.** The `payments` table needed both a `supplier_id` and a `customer_id` column, with a rule that only one could be filled in.

The single-**party** model is the usual approach in accounting and ERP systems. One record describes a business, and its *type* says what role it plays.

2. What “payment terms” means

Payment terms are the agreed rules for **when you have to pay** for goods, or when the other side has to pay you. They are an agreement between the two businesses and are often printed on the invoice.

Term	Plain meaning	Example
Net 30	Pay the full amount within 30 days of the invoice date.	Invoice 1 Oct → pay by 31 Oct
Net 15	Same idea, but within 15 days .	Invoice 1 Oct → pay by 16 Oct
Cash on Delivery (COD)	Pay when the goods arrive . No credit.	Pay the delivery person
Advance / Prepaid	Pay before the goods are sent.	Pay first, then they dispatch

2/10 Net 30

2% discount if paid within 10 days; otherwise the full amount is due within 30 days.

Rs 10,000 → pay Rs 9,800 by day 10

An everyday comparison

It's like a shop that lets a regular customer "pay at the end of the month." That agreement, "end of the month," is the payment term.

How payment terms work in this app

- It is a **free-text note** on each party, stored in the `payment_terms` column (up to 255 characters). You can type anything, such as "Net 30" or "COD".
- It shows as small grey text under the supplier's name in the **Purchase** screen's supplier picker, as a reminder when you record a purchase.
- You can now set and edit it in the **Parties** add/edit form. The old Suppliers form did not show it.
- It is **for information only**. The app does not calculate due dates, send reminders or flag late payments from it. That could be added later.

3. Before and after: the data model

BEFORE

suppliers: name, contact person, PAN/VAT, address, phone, email, payment terms, status, notes

customers: name, phone, address, email, status

purchases.**supplier_id** → suppliers

sales.**customer_id** → customers

batches.**supplier_id** → suppliers

purchase_returns.**supplier_id** → suppliers

payments.**supplier_id** + payments.**customer_id**



AFTER

parties: name, **party_type**, contact person, PAN/VAT, address, phone, email, payment terms, status, notes

purchases.**party_id** → parties (required)

sales.**party_id** → parties (optional: walk-in)

batches.**party_id** → parties (optional)

purchase_returns.**party_id** → parties (required)

payments.**party_id** → parties (one column)

The new `parties` table

Column	Type	Notes
<code>id</code>	uuid, primary key	Random by default. Migrated rows kept their old supplier/customer IDs .
<code>organization_id</code>	uuid, not null	Belongs to one organization; deleted along with it (cascade).
<code>name</code>	varchar(255), not null	

party_type	enum party_type	SUPPLIER CUSTOMER BOTH ; default BOTH .
contact_person	varchar(255)	A named person at the business.
pan_vat_number	varchar(50)	
address	text	
phone	varchar(30)	
email	varchar(255)	
payment_terms	varchar(255)	See section 2.
status	boolean, default true	Active or inactive.
notes	text	
created_at	timestamp, default now()	

The proposal this was based on used a plain `varchar` for the party type. We used a Postgres `enum` instead, as the rest of the schema does for fixed lists such as payment type and batch status. The database then rejects any value other than the three allowed ones.

4. How party types work

Type	Appears in the Purchase supplier picker?	Appears in the Sales customer picker?
SUPPLIER	Yes	No
CUSTOMER	No	Yes
BOTH	Yes	Yes

The rule lives in one place, the party controller: asking for suppliers returns `SUPPLIER + BOTH`, and asking for customers returns `CUSTOMER + BOTH`. To turn an existing supplier into “both”, edit it on the Parties page and change its type. You don’t need to create it again.

5. Every file that changed

Added · **Modified** · **Deleted**. All paths are relative to the `ims` project folder.

Database schema (`src/db/schema/`)

File	Status	What changed
<code>parties.ts</code>	Added	The new <code>parties</code> table and its relations (batches, purchases, sales, payments).
<code>enums.ts</code>	Modified	Added the <code>partyTypeEnum</code> (<code>party_type</code>) enum.
<code>suppliers.ts</code> , <code>customers.ts</code>	Deleted	Replaced by <code>parties.ts</code> .
<code>index.ts</code>	Modified	Exports <code>parties</code> instead of <code>suppliers / customers</code> .
<code>organizations.ts</code>	Modified	Relations: <code>parties: many(parties)</code> replaces the two old relations.
<code>purchases.ts</code>	Modified	<code>supplierId</code> → <code>partyId</code> (required); relation <code>supplier</code> → <code>party</code> .
<code>sales.ts</code>	Modified	<code>customerId</code> → <code>partyId</code> (optional); relation <code>customer</code> → <code>party</code> .
<code>batches.ts</code>	Modified	<code>supplierId</code> → <code>partyId</code> ; relation <code>supplier</code> → <code>party</code> .
<code>returns.ts</code>	Modified	<code>purchaseReturns.supplierId</code> → <code>partyId</code> .
<code>payments.ts</code>	Modified	<code>supplierId</code> + <code>customerId</code> → a single <code>partyId</code> ; the direction is still given by <code>purchaseId / saleId</code> .

Validation (`src/lib/validation/`)

File	Status	What changed
<code>party.ts</code>	Added	Create/update/list/stats schemas for parties, including <code>partyType</code> and a <code>type</code> list filter.
<code>supplier.ts</code> , <code>customer.ts</code> , <code>schema.ts</code>	Deleted	Only used by the old supplier/customer code.
<code>purchases.ts</code>	Modified	<code>supplierId</code> → <code>partyId</code> (create and update).
<code>sales.ts</code>	Modified	<code>customerId</code> → <code>partyId</code> (create and update). See section 9.
<code>batches.ts</code>	Modified	Per-line override <code>supplierId</code> → <code>partyId</code> .

Controllers (`src/controller/`)

File	Status	What changed
------	--------	--------------

party/controller.ts	Added	List (search, paging, type filter), get, create, update, delete and stats for parties.
supplier/controller.ts , customer/controller.ts	Deleted	Merged into the party controller.
purchase/controller.ts	Modified	Writes partyId on purchases and batches.
sales/controller.ts , sales/batchesWise/conteoller.ts	Modified	Writes partyId on sales (create and update).
batches/controller.ts	Modified	Joins parties ; returns party and partyName instead of supplier and supplierName .
dashboard/controller.ts	Modified	Recent transactions join parties for both sales and purchases.

API routes (src/app/api/)

File	Status	What changed
parties/route.ts , parties/[id]/route.ts , parties/stats/route.ts	Added	See section 6.
supplier/* , customers/* (6 routes)	Deleted	Replaced by /api/parties .
purchases/route.ts , purchases/[id]/route.ts	Modified	Load the party relation instead of supplier .
sales/route.ts , sales/[id]/route.ts	Modified	Load the party relation instead of customer .

Pages (src/app/pharma/)

File	Status	What changed
parties/page.tsx	Added	New Parties page. See section 7.
suppliers/page.tsx , customers/page.tsx	Deleted	Replaced by the Parties page.
_components/Sidebar.tsx	Modified	The “Contacts” group (Customers, Suppliers) is replaced by a single Parties link.
purchase/page.tsx	Modified	Loads /api/parties?type=SUPPLIER ; sends partyId ; reads purchase.party .
sales/page.tsx	Modified	Loads /api/parties?type=CUSTOMER ; sends partyId ; a customer added from the sales screen is saved as CUSTOMER .

batches/page.tsx , inventory/page.tsx	Modified	Read partyName and party instead of supplierName and supplier .
--	----------	---

Other

File	Status	What changed
src/db/seed/run-suppliers-only.ts	Modified	Seeds the three sample suppliers into parties as SUPPLIER .
scripts/migrate-to-parties.sql	Added	The one-time data migration (section 8).
backups/before-parties.sql	Added	A full database backup taken before the migration. Do not commit it , because it contains real data.

6. New API reference

All endpoints require the usual login (`getAuth`) and only ever see the logged-in user's organization.

Endpoint	Purpose
<code>GET /api/parties</code>	List parties. Query: <code>page</code> (default 1), <code>limit</code> (default 20, max 100), <code>search</code> (name), <code>type = SUPPLIER CUSTOMER</code> (optional; each also includes <code>BOTH</code>). Returns <code>{ parties, pagination }</code> . Cached for 5 minutes.
<code>POST /api/parties</code>	Create a party. Body: <code>name</code> (required), <code>partyType</code> (default <code>BOTH</code>), and optionally <code>contactPerson</code> , <code>panVatNumber</code> , <code>address</code> , <code>phone</code> , <code>email</code> , <code>paymentTerms</code> , <code>status</code> , <code>notes</code> . Returns <code>{ party }</code> with status 201.
<code>GET /api/parties/:id</code>	One party. Returns <code>{ party }</code> , or 404.
<code>PATCH /api/parties/:id</code>	Partial update with any of the create fields. Returns <code>{ party }</code> .
<code>DELETE /api/parties/:id</code>	Delete a party. Fails if purchases or returns still reference it (the database protects them).
<code>GET /api/parties/stats</code>	Counts. Optional <code>type</code> . Returns <code>{ stats: { totalCount, activeCount, inactiveCount, supplierCount, customerCount } }</code> .

Field renames seen by the frontend

Where	Old	New
Create/update purchase body	<code>supplierId</code>	<code>partyId</code>
Create/update sale body	<code>customerId</code>	<code>partyId</code>
Purchase response	<code>purchase.supplier</code>	<code>purchase.party</code>
Sale response	<code>sale.customer</code>	<code>sale.party</code>
Batch list / expiry list	<code>supplier</code> , <code>supplierName</code>	<code>party</code> , <code>partyName</code>

7. User-interface changes

- **Sidebar:** one **Parties** link replaces the Contacts → Customers / Suppliers group.
- **Parties page** (`/pharma/parties`):
 - Tabs: **All Parties**, **Suppliers**, **Customers**. The stat cards (total, active, inactive) follow the selected tab.
 - A new **Type** column with coloured badges.
 - The add/edit form has a **Party Type** selector (Supplier / Customer / Both), which starts on the current tab's type, plus a **Payment Terms** field.
 - Search, status filter, paging, the click-to-toggle Active/Inactive badge and delete confirmation all work as they did on the old pages.

- **Purchase** and **Sales** screens look the same. Their on-screen labels still say "Supplier" and "Customer", because those words describe the role on that screen.

8. Database migration: what was done

Your database is a hosted **Neon** Postgres 18 instance. The steps, in order:

1. **Pre-check, read only.** Confirmed that the old tables and columns existed, `parties` did not exist yet, no IDs clashed between suppliers and customers, and no business appeared in both tables.
2. **Backup.** A full `pg_dump` of all 18 tables was saved to `backups/before-parties.sql`.
3. **Migration.** `scripts/migrate-to-parties.sql` was run in **one transaction**, so if any statement had failed, nothing would have changed. It:
 - created the `party_type` enum and the `parties` table;
 - copied every supplier (as `SUPPLIER`) and customer (as `CUSTOMER`) across, **keeping their IDs** so existing links stayed valid;
 - added `party_id` to purchases, sales, batches, purchase_returns and payments, filled it from the old column, added the foreign keys, then dropped the old columns;
 - dropped the old `suppliers` and `customers` tables.

4. **Check afterwards, read only.** Results:

Check	Before	After	Result
Suppliers → parties (<code>SUPPLIER</code>)	4	4	OK
Customers → parties (<code>CUSTOMER</code>)	4	4	OK
Purchases linked to a party	19 / 19	19 / 19	OK
Batches linked to a party	21 / 21	21 / 21	OK
Sales linked to a party	7 / 13	7 / 13	OK. The other 6 are walk-in sales with no customer, the same as before.
Links pointing at a party that doesn't exist	n/a	0	OK
Old <code>suppliers</code> / <code>customers</code> tables	present	removed	OK

9. Problem found and fixed afterwards

An open editor tab overwrote a file

About 30 minutes after the changes, `src/lib/validation/sales.ts` was saved from an old editor tab. The copy it saved used `customerId` and was missing `freightCharges`, `vatRefund` and `roundOff`, which caused 16 type errors in the sales controllers and `helper.ts`.

Fix: the file was restored from the last git commit and `customerId` was changed to `partyId` again. The typecheck passes, and no other file was affected. The overwritten copy was kept at `%TEMP%\sales.overwritten.ts` in case it contained anything you wanted.

To prevent it: close all editor tabs **without saving**, then run "TypeScript: Restart TS Server" in VS Code.

10. Things kept on purpose

- `purchases.supplier_invoice_number` is the invoice number printed on the supplier's bill, not a link to a table, so it keeps its name.
- The payment direction values `PAID_TO_SUPPLIER` and `RECEIVED_FROM_CUSTOMER` describe which way money moves, which still makes sense with parties.
- The labels "Supplier" and "Customer" on the Purchase and Sales screens, and some internal variable names such as `customers` in the sales page, describe the *role* on that screen.
- The seed script's name, `run-suppliers-only.ts` / `db:seed:suppliers`, is unchanged. It still seeds suppliers, now into `parties`.

11. Remaining steps and checklist

Status

Code changes: done, and the typecheck passes · Database migration: done and verified · Git: **nothing committed or pushed**, as you asked.

#	Step	Expected result
1	Run <code>pnpm db:push</code>	"No changes detected", or only minor ones. Answer no if it offers to drop or truncate anything.
2	Run <code>pnpm dev</code> . If old routes cause errors, delete the <code>.next</code> folder first.	The app starts.
3	Open Parties from the sidebar.	8 parties; the Suppliers and Customers tabs show 4 each.
4	Create and edit a purchase.	The supplier picker lists suppliers and "both" parties; old purchases still show their supplier.

5	Create and edit a sale; add a new customer from the sale screen.	The new customer appears on the Parties page as Customer .
6	Open the Dashboard, Batches and Inventory pages.	Party names appear where supplier and customer names used to.
7	Add <code>backups/</code> to <code>.gitignore</code> .	The backup can't be committed by accident.
8	When happy, commit the changes yourself.	—

12. How to undo (rollback)

- **Database:** restore `backups/before-parties.sql` into an empty database, or use Neon's **point-in-time restore** (Branches → Restore) to go back to a moment after the backup (taken 28 September 2026 at 13:22) and before the migration was run.
- **Code:** nothing is committed, so `git restore .` plus deleting the new files (`src/db/schema/parties.ts` , `src/lib/validation/party.ts` , `src/controller/party/` , `src/app/api/parties/` , `src/app/pharma/parties/` , `scripts/`) returns the code to the last commit. The code and the database must be rolled back **together**.